SAPPHIRE PROGRAMMING LANGUAGE

Complete Developer Coding Manual, Deep Learning, Concurrency & AI Agent Reference

Version 1.0.0 (Official Manual) | Sapphire Language Architecture Team

Chapter 1: Executive Overview & Design Philosophy

Sapphire is an autonomous-first, statically and gradually typed programming language designed specifically for **PC Automation**, **Deep Neural Network Training**, and **Autonomous AI Agent Systems**. Modern developer workflows are severely fragmented across Python (for ML and AI scripting), Rust/C++ (for kernel performance and memory safety), JavaScript (for web APIs and async events), and Bash/PowerShell (for process orchestration). Sapphire collapses this toolchain into a single, unified runtime.

The Core Autonomous Pipeline

Data → Training → Model → Reasoning → Memory → Planning → Tool Use → Autonomous Execution

Key Architectural Pillars:

- **Colorless Concurrency:** Lightweight green fibers eliminate async function coloring and callback hell. Native parallel blocks allow concurrent execution without async/await.
- **Stream & Process Piping:** First-class shell commands and the |> pipe operator stream data effortlessly between OS processes, arrays, and functions.
- **Embedded Deep Learning Stack:** Native tensors, automatic differentiation (autograd), neural layers, and multi-worker distributed training directly in stdlib (ml module).
- **Autonomous Agent Engine:** Vector/key-value memory, 4-step DAG planners, ReAct reasoning loops, dynamic tool registries, and permission policies (agent module).
- **System & Hardware Automation:** Direct hardware telemetry, mouse/keyboard simulation, GUI alerts, and desktop notifications without third-party packages.

Chapter 2: Language Syntax, Variables & Types (.sp)

Sapphire scripts use the .sp file extension. Sapphire supports both dynamic type inference and optional static type annotations.

```
// Variable declarations and dynamic type inference
let agent_name = "Sapphire-Omni";
let max_retries = 5;
let threshold = 0.985;
let is_autonomous = true;

// Composite collections
let features = [0.12, 0.45, 0.88, 0.93];
let system_config = {
    "device": "cuda:0",
    "batch_size": 64,
    "learning_rate": 0.001,
    "model_type": "mlp"
};

// String interpolation with formatting
print("Agent {agent_name} configured with device {system_config['device']}");
```

Chapter 3: Control Flow & Loops

Sapphire provides standard structured control flow: conditional branching (if/else), counted loops (for/in), and condition-controlled loops (while).

```
// Conditional branching
if (system_config["batch_size"] > 32) {
    print("■ High throughput batching enabled.");
} else {
    print("■■ Standard batching mode.");
}

// For-in loop over collections
for (val in features) {
    print("Processing feature value: {val}");
}

// While loop with break/continue
let counter = 0;
while (counter < 10) {
    counter = counter + 1;
    if (counter == 3) { continue; }
    if (counter == 8) { break; }
}
```

Chapter 4: Functions, Closures & Pipe Operator

Functions in Sapphire are declared with the fn keyword and can be passed as first-class values. The **pipe operator (|>)** passes the expression on the left as the first argument to the function on the right.

```
// Function definition with default & typed arguments
fn compute_loss(predicted, actual) {
    let diff = predicted - actual;
    return diff * diff;
}

// Pipe Operator Pipeline Flow
fn normalize(data) {
    return ml.dataset.normalize(data);
}

fn evaluate(dataset) {
    print("Dataset evaluated successfully.");
    return dataset;
}

// Flow data through pipeline using |>
let raw_data = [10.0, 20.0, 30.0, 40.0];
let clean_data = raw_data |> normalize |> evaluate;
```

Chapter 5: Colorless Concurrency (parallel blocks)

Sapphire eliminates function coloring. Any code block wrapped in parallel { ... } runs task branches concurrently across worker threads.

```
// Structured Concurrency in Sapphire
parallel {
    // Branch 1: Sensor telemetry
    let stats = os.system_info();
    print("RAM Usage: {stats.ram_percent}%");

    // Branch 2: Web API polling
    let response = http.get("https://httpbin.org/get");
    print("API Status: {response.status_code}");

    // Branch 3: Disk workspace scan
    let files = fs.list_dir(".");
    print("Discovered {files.length} workspace files.");
}
print("■ All parallel branches synchronized.");
```

Chapter 6: Deep Learning & Tensor Standard Library (ml)

Sapphire includes a native deep learning framework with tensors, automatic differentiation, neural layers, loss functions, optimizers, and distributed data-parallel training.

```
// 1. Tensors & Matrix Multiplication
let A = ml.tensor([[1.0, 2.0], [3.0, 4.0]]);
let B = ml.tensor([[5.0, 6.0], [7.0, 8.0]]);
let C = A.matmul(B);
print("Matrix Product: {C}");

// 2. Autograd & Automatic Differentiation
let x = ml.autograd.variable(3.0);
let tape = ml.autograd.tape();
tape.watch(x);
let y = x * x + 2.0 * x + 1.0;
let dy_dx = tape.gradient(y, x);
print("d/dx (x^2 + 2x + 1) at x=3: {dy_dx}"); // Expected: 8.0

// 3. Neural Network Architecture & Model Creation
let model = ml.model.mlp([16, 64, 32, 2], "relu");

// 4. Distributed Multi-Worker Training
let dataset = ml.dataset.random(1000, 16, 2);
let split = dataset.split(0.8);

let training_results = ml.train.fit(
    model,
    split["train"],
    ml.loss.cross_entropy,
    ml.optim.adam(0.001),
    epochs=5,
    batch_size=32,
    n_workers=4,
    val_dataset=split["val"]
);
print("Final Validation Loss: {training_results.final_loss}");
```

Chapter 7: Autonomous Agent Architecture (agent)

The agent module provides production-grade agent primitives: vector memory, 4-step DAG planners, tool registries, security permissions, and autonomous loop engines.

```
// 1. Memory Store (Short & Long Term Semantic Recall)
agent.memory.remember("model_checkpoint", "v1.2.0-prod");
agent.memory.remember("target_accuracy", 0.982);
let val = agent.memory.recall("model_checkpoint");

// 2. Tool Registration & Execution
agent.tools.register("disk_cleanup", "Deletes temporary cache files", fn(path) {
    let removed = fs.remove(path);
    return "Cleanup status: {removed}";
});

let tool_result = agent.tools.execute("disk_cleanup", "./tmp");
print("Tool Execution Result: {tool_result}");

// 3. Autonomous Loop Execution
let goal = "Train policy network, deploy to production, and alert administrator.";
let report = agent.autonomy.run_loop(goal, max_steps=5);
print("Agent Autonomous Loop Finished: {report['finished']}");
```

Chapter 8: System Automation & OS Control (os, fs, gui)

```
// Hardware Telemetry
let sys_info = os.system_info();
print("CPU: {sys_info.cpu_usage_percent}%, RAM: {sys_info.ram_percent}%");

// GUI Automation & Toast Alert
gui.alert("System Health Normal", "Sapphire Monitor");
os.notify("Sapphire Agent", "All systems operational.");
```

Chapter 9: End-to-End Autonomous Autobot Script (.sp)

Below is a complete, production-ready autonomous PC autobot written entirely in Sapphire:

```
// 05_full_pc_autobot.sp — Autonomous PC Monitor & Remediation Agent
fn pc_autobot() {
    print("■ SAPPHIRE AUTONOMOUS BOT INITIALIZING...");

    // Phase 1: Sense System Telemetry
    let stats = os.system_info();
    print("RAM Used: {stats.ram_percent}%, CPU: {stats.cpu_usage_percent}%");

    // Phase 2: Parallel Perception
    parallel {
        let http_check = http.get("https://httpbin.org/get");
        let workspace_files = fs.list_dir(".");
        print("Indexed {workspace_files.length} workspace items.");
    }

    // Phase 3: AI Cognitive Evaluation
    let prompt = "System RAM is {stats.ram_percent}%. Evaluate health and recommend action.";
    let ai_decision = ai.prompt(prompt);
    print("■ AI Decision: {ai_decision}");

    // Phase 4: Memory Persistence
    agent.memory.remember("last_pc_health_eval", ai_decision);

    // Phase 5: Autonomous Action & Notification
    os.clip_write("PC Diagnostic: RAM {stats.ram_percent}%");
    os.notify("Sapphire Autobot", "Autonomous PC Health Check Completed!");
}

pc_autobot();
```

Chapter 10: Emerald Developer Studio IDE Reference

Emerald Developer Studio (launched via sapphire studio or Emerald_Studio.exe) is the official graphical integrated development environment for Sapphire.

- **Built-in .sp Code Editor:** High-performance syntax-aware code editor with auto-indentation and file management.
- **1-Click Sapphire Tool Builder:** Interactive dialog wizard to rapidly scaffold and register new autonomous agent tools.
- **Live Compiler Terminal:** Integrated execution terminal with live stdout/stderr capture and ANSI color formatting.
- **Hardware Telemetry Dashboard:** Real-time CPU load, RAM usage, and NVIDIA CUDA GPU VRAM monitoring.
- **Agent & Memory Inspector:** Live state inspector for agent working memory, vector embeddings, and active execution DAGs.

Chapter 11: Standard Library Reference Table

Module	Function / API Signature	Description
`ml.tensor`	`ml.tensor(data, dtype, device)`	Creates multi-dimensional tensor array.
`ml.autograd`	`ml.autograd.tape()`	Context manager for automatic differentiation.
`ml.train`	`ml.train.fit(model, ds, loss, opt, ...)`	Data-parallel multi-threaded neural trainer.
`ml.gpu`	`ml.gpu.info() / to_device(t, dev)`	CUDA/GPU hardware device telemetry & tensors.
`ai.prompt`	`ai.prompt(query: str) -> str`	LLM reasoning via local Ollama or Groq API.
`agent.memory`	`agent.memory.remember(k, v)`	Stores vector/key-value semantic knowledge.
`agent.tools`	`agent.tools.register(name, desc, fn)`	Registers runtime executable tool for agents.
`agent.autonomy`	`agent.autonomy.run_loop(goal, steps)`	Runs autonomous ReAct execution loop.
`os.system_info`	`os.system_info() -> Map`	Returns live CPU, RAM, and Disk metrics.
`fs.write`	`fs.write(path, data) -> Bool`	Writes UTF-8 text or binary content to file.
`http.get`	`http.get(url, headers) -> Map`	Performs HTTP GET returning status & body.